

Galaxy CodeVital — Technical Call Talking Points

Doc Re-Read Notes (Don't Miss These)

- **Whisper was already evaluated** by their team in the 4.1 literature review alongside OpenCV and wav2vec. Don't present it as new — present it as the right tool they identified but didn't fully implement correctly (specifically: speaker diarization was missing)
- **The domain LLM is primarily for the AI Tutor Avatar** — not just general analytics. From the SR&ED doc: "They do not represent tutor personality faithfully." The LLM needs to *sound like a specific tutor*, not just answer tutoring questions generically. This changes what the RAG knowledge base needs to contain
- **Synthetic tutoring conversations** are explicitly mentioned in the 2025 doc as a possible training data approach — worth acknowledging
- **Their existing stack already has BullMQ/RabbitMQ** — the post-session batch pipeline maps directly onto infrastructure they've already built
- **They already tried tutor speech analysis** (Attempt 7) and LLM-based tutor style clustering (Attempt 8) — both failed due to noise and small datasets. Our transcript-based TVV approach needs to be framed as *why it avoids those failure modes*, not just what it does
- **Cosine similarity matching already failed** (Attempt 9) — they know this. Don't just say "cosine similarity failed," explain specifically why vocabulary similarity $\neq$ compatibility
- **RL matching was tried and failed** — not enough labeled outcome data. Confirms our "defer Two-Tower, start with archetypes" logic

How to Open

Don't pitch yourself. Open with diagnosis.

"I went through both documents carefully — the SR&ED narrative and the 2025 roadmap. I have a clear picture of where things stalled and I have some concrete thoughts on how to move each piece forward. Happy to walk through them."

Then let them direct — ask if they want to start with the active blockers or the 2025 roadmap items. Either way, you're ready.

Talking Point #1 — Learning Velocity: Fix the Input Before the Model

The diagnosis (say this): "Looking at the BCV failure, the scoring formula wasn't the problem — the upstream signals were too noisy for any model to stabilize on. Every version tried to fix downstream scoring while the feature extraction was still unreliable. I'd flip the approach entirely."

Critical implementation detail — Speaker Diarization: Whisper alone cannot distinguish between multiple speakers. It transcribes audio but doesn't know who said what. Before any velocity computation is possible, you need speaker diarization — separating the audio stream into per-speaker segments.

The standard tool is **pyannote.audio**, which runs as a pre-processing step before Whisper:

- pyannote segments the audio: "Speaker 1: 0:00–0:45, Speaker 2: 0:45–1:20..."
- You then map Speaker 1 → Tutor, Speaker 2 → Student based on role (tutor speaks first, or via voice enrollment)
- For group sessions with multiple children, each child gets their own speaker ID — their transcript segments are extracted and embedded independently
- This means Learning Velocity is computed per-student, per-session, from only *that student's* speech — not the full session transcript

This is a non-trivial engineering step but it's solved infrastructure. pyannote + Whisper is the community standard pipeline for multi-speaker transcription.

The solution (A + C):

Phase 1 — Transcript-Based Semantic Progression:

- Run pyannote diarization → Whisper transcription on each speaker segment
- Embed each student's per-session transcript using SentenceTransformers
- Learning Velocity = slope of cosine distance between consecutive session embeddings over time
- Reproducible, computable from session 3 onward, zero dependency on video signal quality
- I've built this SentenceTransformers pipeline at production scale — 100K+ records, sub-second retrieval

Phase 2 — Intra-Session Granularity:

- Whisper gives word-level timestamps — chunk student speech into 5-minute windows
- Embed each chunk → measure concept progression within a single session
- Finer-grained signal, no new tooling required

Cold-Start (C):

- Structured intake question sequence in session 1 → embed responses → initial LV estimate
- One session is enough to initialize; Bayesian update from there

If they push back with "we lose the video engagement signals": *"You don't lose them — you defer them. They already tried video signals in 2024 and the noise was unmanageable. Get velocity stable on transcripts first, then layer video back in as a confidence booster once the baseline is solid."*

Talking Point #2 — Teaching Velocity: Transcripts + Archetypes + Scoring

Why transcript-only avoids their 2024 failure modes: Attempt 7 (tutor speech analysis) failed because acoustic signals from video — pitch, ambient noise, inconsistent recording quality — were too noisy. Attempt 8 (LLM clustering) failed because embeddings were inconsistent on small datasets. Transcript-based features are deterministic, reproducible, and don't degrade with audio quality issues.

The solution (A + C combined):

TVV Feature Set — going beyond just vocab and Q&A:

Reading their docs carefully, they tried to capture: explanation pace, complexity, question cadence, adaptability, patience, tone, repetition, and interaction density. Our transcript features should map to these:

- **Readability (Flesch-Kincaid grade level)** → complexity of explanations
- **Question density** → questions per minute → interaction style
- **Explanation length** → average words before asking for student response → pace of delivery
- **Type-token ratio** → vocabulary richness → conceptual coverage
- **Repetition rate** → how often the tutor repeats or rephrases → adaptability signal
- **Student correction frequency** → how often tutor corrects errors → teaching responsiveness
- **Speech rate + pause frequency** (from Whisper timestamps) → pacing
- **Turn-taking ratio** → tutor talk time vs. student talk time → interactive vs. lecture style

This directly covers the 20+ feature TVV they attempted but couldn't stabilize on from video. All of these are computable from clean transcript + timestamps.

Concrete Scoring System: Each feature gets normalized to a 0-1 scale, then weighted into a

composite TVV score per session. Weights are initialized equally and updated over time as session outcome data accumulates:

```
TVV_score = w1×(speech_rate_normalized) + w2×(question_density_normalized)
           + w3×(vocab_richness) + w4×(explanation_length) + ...
```

This gives you a point estimate per session — the *average* TVV score across a tutor's sessions becomes their profile. Confidence interval widens with fewer sessions, narrows with more.

Behavioral Archetype Clustering on top of scoring:

- Run K-Means (k=4-5) on TVV feature vectors across all tutors
 - Each archetype gets a label: "Methodical Explainer," "Socratic Questioner," "Fast-paced Driller," "Adaptive Coach"
 - The archetype is the *interpretable label* for parents
 - The TVV score is the *quantitative signal* for the matching engine
 - Both are needed — archetypes for UX, scores for ML
-

Talking Point #3 — Matching Engine: Hard Filter + Archetype Ranking

The diagnosis (say this): "The cosine similarity approach failed because it matched teaching vocabulary, not teaching compatibility. Two people can use similar words and be completely incompatible teachers for a given student."

The solution (C first, B and A over time):

Day 1 — Hard Filter + Archetype Ranking:

- Stage 1: Rule-based hard filter — grade level, subject, location/availability, price range
- Stage 2: Within that filtered set, rank tutors by compatibility between student's learning pattern and tutor's teaching archetype
- Works immediately, no labeled outcome data needed
- This is the same two-stage retrieval architecture I built at Humanitarians AI with Chroma DB — hard constraints first, then embedding-based ranking

Over time — Collaborative Filtering (B):

- As session history accumulates: "students similar to this student learned well with tutor X"
- Start collecting outcome signals now via post-session surveys so this trains itself
- Natural self-improvement — gets smarter with every completed session

What about the Two-Tower model (A)?

- Worth understanding: it's two separate neural encoders — one for student LV features, one for tutor TV features — trained jointly on session outcome labels to learn a shared compatibility space
 - The reason to defer it: it requires labeled outcome data to train (engagement scores, grade improvements, parent ratings) which doesn't exist yet
 - But it's the right long-term upgrade once you have 6+ months of session history — and C is literally the infrastructure it will need
-

Talking Point #4 — Domain LLM: What It Actually Does and Why They Want It

Why they want a domain-specific LLM — reading the docs carefully: Their SR&ED narrative is explicit: "They do not represent tutor personality faithfully." The primary use case is the **AI Tutor Avatar** — a version of a specific tutor that students can practice with between live sessions. Generic GPT can't do this because it doesn't know how Tutor Sarah explains fractions, what examples she uses, or how patient she is with repeated questions. They need an LLM that speaks as a *specific tutor*, grounded in that tutor's actual session history.

Secondary use cases: session summarization, teaching style classification, parent-facing Q&A about their child's progress.

What the RAG knowledge base actually contains:

- That tutor's session transcripts (how they actually explain concepts)
- Their TVV feature profile (teaching style markers)
- Subject-specific curricula and concept maps
- Student session history and LV progression
- Parent feedback and session notes

What RAG achieves for each use case:

AI Tutor Avatar:

- Parent or student asks: "Can you explain long division the way Tutor Sarah does?"
- RAG retrieves: Sarah's past transcripts where she taught long division
- Qwen3 8B generates: a response in Sarah's actual teaching style, grounded in her real explanations — not hallucinated

Session Summarization:

- After each session, RAG retrieves the transcript + student's LV history
- Generates: structured summary with progress notes, concepts covered, recommended focus areas

Parent Q&A:

- Parent asks: "Is my child improving in math?"
- RAG retrieves: student's LV trajectory, recent session summaries, tutor feedback
- Generates: grounded, specific answer — not a generic "your child is doing great"

The replace "train from scratch" argument: "The 2025 doc mentions fine-tuning vs. training from scratch as an open question, and even suggests synthetic tutoring conversations as training data. RAG with Qwen3 8B locally deployed solves the core problem — personality-faithful, privacy-safe responses — in weeks, not months. Fine-tuning on real tutor transcripts is the next layer once you have enough session data."

The solution:

Week 1-2 — Prompt Engineering + Structured Output:

- Session summarization, style classification — well-engineered prompts may be sufficient immediately
- Deployable in days, reveals what actually needs fine-tuning

Week 2-4 — Local RAG Pipeline:

- Deploy Qwen3 8B locally via Ollama (dev) or vLLM (production)
- Knowledge base in Chroma DB, all data on CloudVital servers
- Why Qwen3 8B: 4.8% hallucination rate, Apache 2.0, 5-6GB VRAM, 131K context window
- **Critical:** do NOT use reasoning models (DeepSeek R1 etc.) for RAG — hallucination rate jumps to 14.3% because chain-of-thought fills gaps beyond retrieved context

Month 1-2 — Fine-tune on Tutor Transcripts:

- QLoRA fine-tuning on accumulated session transcripts
- Tasks: teaching style classification, difficulty assessment, personality-faithful response generation
- Unsloth for efficiency: fits on 8-12GB VRAM, 2-5× faster than standard fine-tuning

Redirect federated learning: "Federated learning is the right long-term direction but complex infrastructure for your current stage. The RAG architecture solves the core privacy requirement structurally — student data never leaves your servers. Pair that with a data anonymization pipeline stripping PII and you're covered for regulatory compliance without the operational overhead."

Concrete experience to cite:

HyperAnalytics (healthcare — HIPAA):

- Worked with protected health information — applied PII stripping before any model processing
- AWS S3 is HIPAA-compliant by default with the right configuration (encryption at rest, access logging, BAA in place)
- Designed the data pipeline so patient identifiers never touched model inputs

Future Focused Education (EdTech — parental data concerns):

- Parents were explicitly concerned about student data leaking to large AI companies
- Deployed local model servers — no external API calls, all inference on-premises
- This is exactly the architecture Galaxy needs — and I've already implemented it in an EdTech context specifically

The full privacy stack to propose:

1. Local RAG deployment — no data leaves CloudVital servers
 2. PII anonymization pipeline — names, faces, voices pseudonymized before model processing
 3. Access logging — every retrieval and generation logged locally for audit
 4. Covers PIPEDA (Canada) and FERPA (US) requirements structurally
-

Talking Point #7 — Session Analytics: Question the Real-Time Assumption

The reframe (say this): "Before solving real-time analytics, I'd question whether it's actually necessary at this stage. The outputs that matter — Learning Velocity scores, session summaries, teaching style analysis, match quality feedback — are all things parents and tutors consume after the session. A tutor in the middle of a live session doesn't need an AI dashboard to tell them engagement dropped. They can see the student."

The better approach:

Session ends, \> job queued in BullMQ/RabbitMQ (already in their stack)

- Session ends → job queued in BullMQ/RabbitMQ (already in their stack)
- Full pipeline runs on the complete recording — better analysis quality because the model has full session context, not just live audio chunks
- Report ready within minutes of session end
- Simpler architecture, no latency constraints, higher quality outputs

Why this matters: Real-time in-session analytics is listed as a **Phase 3 item (2026-2027)** in their own roadmap. It's not an active 2025 blocker. Pointing this out shows you read the docs carefully and are thinking about where to spend engineering effort, not just executing the roadmap as written.

If they push back and say real-time is important: Ask what the specific use case is — who sees the live dashboard and what action do they take on it? If it's a supervisor monitoring multiple sessions, that's a real use case worth scoping. If it's the tutor themselves, it's probably noise.

Questions to Ask During the Call

1. "What format are session transcripts currently in — are you using Zoom/Jitsi's built-in transcription or something custom? That'll affect which Whisper variant makes sense."
2. "What does a 'successful match' look like in your current data — do you have any outcome labels yet, or is that still being defined?"
3. "Is the domain LLM primarily meant to power the Avatar feature, or also the in-session analytics summarization? The answer affects the fine-tuning dataset we'd build."
4. "What's the timeline pressure — are you trying to get Learning Velocity into a pilot, or still in R&D phase?"

Their answers to 1 and 4 tell you which proposals to build out first. Answer to 2 tells you when the Two-Tower matching upgrade becomes viable.

What NOT to Lead With

- Federated learning / differential privacy — redirect to RAG as the privacy solution
- Real-time streaming — your lane is batch ML, be honest about that, the event-driven architecture is the right framing
- AI Tutor Avatar — too speculative, no clear ROI timeline
- The 2024 failure analysis — open with "here's how to move forward," not "here's why you failed"

